

GenAI / AI Engineer Learning Summary

Tools: VS Code + Databricks (no other cloud) | Companion docs: [LEARNING_PATH.md](#), [GIT_LEARNING.md](#)

Phase 0 - Environment & Tooling Setup

Python & Dependency Management

- Python 3.12 + uv instead of venv/pip - faster, handles Python versions, envs, and locking in one tool.
- `uv add <package>` auto-creates `.venv` and updates `pyproject.toml` + `uv.lock` (no manual venv create/activate).
- `pyproject.toml` (direct deps) + `uv.lock` (exact resolved versions incl. transitive deps) replace `requirements.txt`.

```
uv init --python 3.12
uv add openai python-dotenv
```

- Gotcha: project lives inside OneDrive - `uv add` can fail with 'Access is denied' (file lock from background sync). Fix: just retry, or pause OneDrive sync briefly if it persists.

VS Code Setup

- Extensions: Python, Pylance, Jupyter, Databricks (official - enables workspace connect, Databricks Connect).

Git & GitHub

- 3 areas: working directory -> staging (`git add`) -> commit history (`git commit`).
- `git status` / `git log --oneline` to inspect state; `git restore <file>` discards uncommitted changes.
- Pager gotcha: `git log/show/diff` open a pager - press 'q' to exit, it isn't frozen.
- Remote workflow: `git remote add origin <url>`, `git push -u origin <local>:<remote-branch>`.
- This project: local master tracks origin/dev-branch (pushes go to dev-branch, not master).

```
git push -u origin master:dev-branch # push + set tracking
git push origin master:some-branch  # one-off push, tracking unchanged
```

- `.gitignore` excludes secrets (`.env`) and machine-specific folders (`.venv`, `.databricks`) from version control.

Databricks Workspace Connection

- Databricks VS Code extension connects to an org workspace; creates `databricks.yml` (bundle config with workspace host).
- Foundation Model APIs: pay-per-token hosted models, OpenAI-compatible endpoint - no separate cloud infra needed.
- Endpoint used throughout: `databricks-claude-opus-5`, called via the standard openai Python client.

```
client = OpenAI(
    api_key=os.environ["DATABRICKS_TOKEN"],
    base_url=f"{os.environ['DATABRICKS_HOST']}/serving-endpoints",
)
```

- SECURITY LESSON: Databricks Playground's 'Get Code' button pastes your PAT token in plaintext into the snippet - never commit that; always load secrets via `.env` (already gitignored) + `python-dotenv`.
- Gotcha: the VS Code Databricks extension can silently rewrite `pyproject.toml` (adding strict version pins to mirror a serverless compute environment) when (re)connecting. Always run `git diff/git status` after connecting, before committing.

Corporate Network Gotcha

- Packages that fetch files over HTTPS (e.g. tiktoken downloading its vocab) can fail with SSL certificate errors on a corporate network doing TLS inspection.
- Fix: install truststore and call `truststore.inject_into_ssl()` before other imports - makes Python trust the Windows OS certificate store.

Phase 1 - LLM Foundations & Prompt Engineering (in progress)

Tokens (not words)

- LLMs process sub-word 'tokens', not whole words. Rough rule of thumb: ~4 characters per token for plain English.
- Common English words = usually 1 token. Punctuation is its own token. A leading space merges into the next token.
- Uncommon/brand/jargon words fragment unpredictably, e.g. 'Databricks' -> ['D','atab','ricks'], 'NTT' -> ['N','TT'].
- Why it matters: API cost and context-window budget are both counted in tokens; domain jargon (company/product names) silently costs more than plain English of the same length.

Context Window

- Max tokens (input + output combined) a model can 'see' at once. Claude Opus 5 has a very large window.
- 'Lost in the middle' effect: models tend to weigh the start/end of a long context more than the middle.

Prompt Structure: Roles

- system: sets persona/behavior/constraints for the whole conversation.
- user: the actual question/instruction.
- assistant: a prior model reply - used to show few-shot examples or continue a multi-turn chat.

Zero-shot vs Few-shot

- Zero-shot: ask directly, no examples - model guesses your desired output format.
- Few-shot: include 1-3 example Q/A pairs as fake prior turns before the real question - model mimics the pattern (e.g. terse single-word answers).

```
{"role": "user", "content": "Classify sentiment: 'I love this!'"},  
{"role": "assistant", "content": "positive"},  
{"role": "user", "content": "Classify sentiment: <real text>"}
```

System Prompt & Chain-of-Thought

- System prompt example: 'You are a terse code reviewer. Answer in at most 2 sentences.' - shapes tone/length regardless of the question.
- Chain-of-thought: asking the model to 'think step by step' gives it room to reason before the final answer - improves accuracy on logic/math-style questions.

Structured Output: Prompting vs Tool/Function Calling

- Prompt-based JSON ('respond ONLY with valid JSON...') usually works but isn't guaranteed - model can add stray text or a markdown fence.
- Tool/function calling is the reliable, industry-standard approach: you define a JSON schema 'tool'; the model returns arguments matching that schema instead of free text.

```
tools = [{  
  "type": "function",  
  "function": {  
    "name": "extract_person",  
    "parameters": {"type": "object",  
      "properties": {"name": {"type": "string"}, "age": {"type": "integer"}},  
      "required": ["name", "age"]}  
  }  
}]
```

```
# response.choices[0].message.tool_calls[0].function.arguments  
# -> guaranteed-parseable JSON string matching the schema
```

Phase 2 - RAG: Chunking Strategies (in progress)

Why chunk at all

- Embedding a whole document as one vector loses precision (can't pinpoint 'the answer is in this one paragraph'); it's also too big to stuff into every prompt.
- Pipeline: chunk -> embed each chunk -> store (vector DB) -> retrieve top-matching chunks per query -> generate answer using those chunks as context.

Step 1: Naive fixed-size chunking (baseline, flawed)

- Just slice text every N characters: `text[i:i+chunk_size]`. Zero awareness of structure.
- Failure modes observed: cuts image markdown links in half, splits table rows, breaks mid-sentence.

Step 2: Structure-aware chunking (split by headings)

- Use a regex to find markdown headings; each section runs from one heading to the next.
- Result: whole sections (e.g. a full pricing table) stay together as one meaningful chunk instead of arbitrary slices.

Step 3: Recursive fallback for oversized sections

- If a section is still too big, split further by paragraph (blank-line boundaries) - never mid-sentence.
- Atomic-unit rule: an image reference (`![alt](url)`) is never split, even if it pushes a chunk slightly over the size limit.
- General principle: split along the biggest safe boundary first (headings > paragraphs > sentences), only going smaller when forced to.

Handling Real PDFs with Embedded Images, Charts & Graphs

- A real PDF has embedded binary images, not markdown links - libraries like PyMuPDF (fitz) or pdfplumber extract text blocks and images separately per page.
- Captioning approach (most common in practice): send each extracted image to a vision-capable LLM (Claude is multimodal) and ask it to describe the image/chart in words.
- The generated caption is plain text -> insert it back into the document flow at the image's original position -> now flows through the same chunk/embed/retrieve pipeline as regular text.
- Keep metadata (page number, image file path) linking each caption back to the original image, so the app can show the real image when citing that chunk.
- Advanced alternative: multimodal embeddings (e.g. CLIP-style) embed images directly into the same vector space as text, skipping the captioning step - more complex, less commonly needed than the captioning approach.